USER GUIDE · VERSION 0.3

AI Orchestration Platform

What it does, how to install it, and how you and your team use it every day, from your first agent to calling it from your own software.

October 2026 · Screens in this guide come from a real installation running in demo mode.

Contents

1. What the platform does
2. Install and start it
3. Create your account and workspace
4. Connect an AI model
5. Add your knowledge (documents)
6. Add tools (your APIs and MCP servers)
7. Build an agent
8. Test, publish and run the agent
9. Follow runs, costs and the audit log
10. Work as a team: invitations and roles
11. Use agents from your own apps (API keys)
12. Put it on a server for your team
13. Everyday operations: backups, updates, passwords
14. Troubleshooting
15. What is ready and what is coming

Words used in this guide

Word	Meaning
Workspace	A private area for one team or project. Agents, documents, keys and runs belong to a workspace. One person can be in several.
Provider	An AI model service: OpenAI, Anthropic (Claude), Google Gemini, or any server that speaks the OpenAI format (for example Ollama, vLLM, LM Studio).
Agent	A reusable AI assistant: instructions, a model, optional tools and knowledge, limits.
Run	One job for an agent: an input message goes in, an answer comes out, and every step is recorded.
Tool	Something the agent may call to act or look things up, such as your order API.
Knowledge base	A collection of your documents that the agent searches before it answers.
Demo mode	A built-in fake model that costs nothing and needs no key, for trying things out. Its answers start with `[DEMO]`.

1. What the platform does

The platform is your own control center for AI assistants ("agents"). You install it on your computer or server, connect the AI models you pay for, and then build agents that your team and your software can use safely.

Build agents without code
Write the instructions, pick a model and a backup model, set limits on tokens, time and cost.

Answer from your documents
Upload PDFs, Word files, Markdown, web pages or text. The agent finds the right passages and shows its sources.

Act through tools
Let the agent call your APIs (orders, CRM, tickets) or tools from MCP servers, with every call checked and logged.

Versions you can trust
Publishing freezes a version. Production uses the published version, so editing a draft never breaks live users. Roll back any time.

Reliable execution
Runs go into a queue and a background worker executes them, with retries, timeouts, cancel, and recovery if a worker crashes.

Team and control
Invite people with roles, see costs and token usage, read the audit log of who changed what.

Use it from anywhere
Call agents from your website, backend or scripts with an API key and a simple HTTP request.

Your data stays with you
It runs on your machine or server. API keys and tool passwords are encrypted. Nothing is sent anywhere except to the model providers you add.

How the parts fit together

There are four programs, started together by Docker:

Part	Job
Web app (port 3000)	The screens you see in the browser.
API (port 8000)	Checks logins and permissions, stores everything, and is what your software calls.
Worker	Runs the agents and reads uploaded documents in the background. You can run several.
MongoDB	The database.

2. Install and start it

You need **Docker**. On Windows or macOS install Docker Desktop and start it. On Linux install Docker Engine with the compose plugin.

1. Unzip `orchestrator-v0.3.x.zip` and open a terminal in the `platform` folder.
2. Start everything with one command. The first start takes a few minutes because it builds the programs.

   ```
   # Windows (PowerShell)
   ./scripts/dev.ps1

   # macOS / Linux
   ./scripts/dev.sh
   ```

   The script creates your settings file `.env` with a fresh encryption key, then starts the four parts.
3. Open **http://localhost:3000** in your browser.

To stop it: `./scripts/dev.ps1 -Down` (Windows) or `./scripts/dev.sh --down`. Your data is kept for the next start.

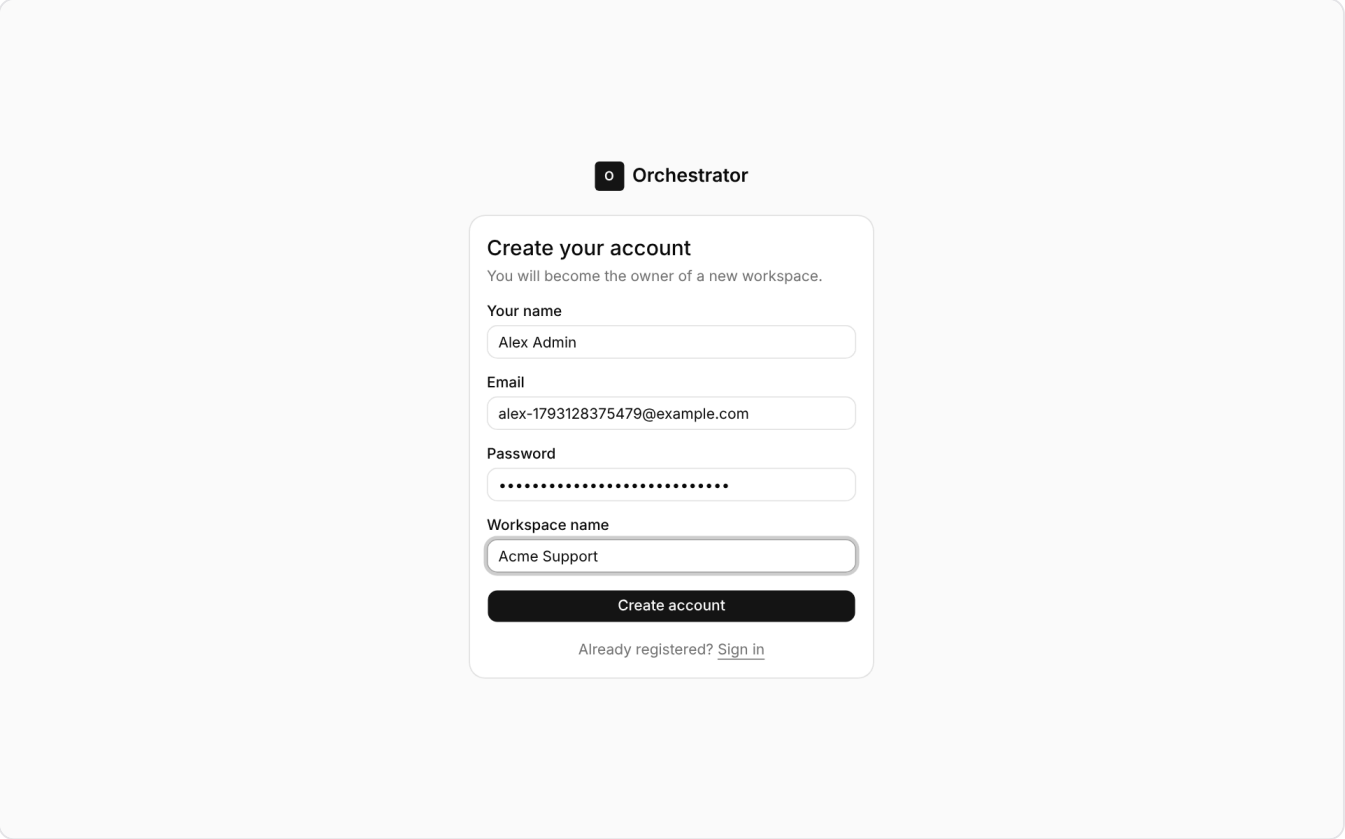
> **Keep the `.env` file safe.** It holds `ENCRYPTION_KEYS`. Without that key the platform cannot read the provider API keys and tool passwords it stored. Never put `.env` into Git.

Useful addresses

Address	What it is
http://localhost:3000	The web app
http://localhost:8000/api/docs	Interactive API documentation (Swagger), for developers
http://localhost:8000/api/v1/health/ready	Health check: answers "ok" when the API and database work

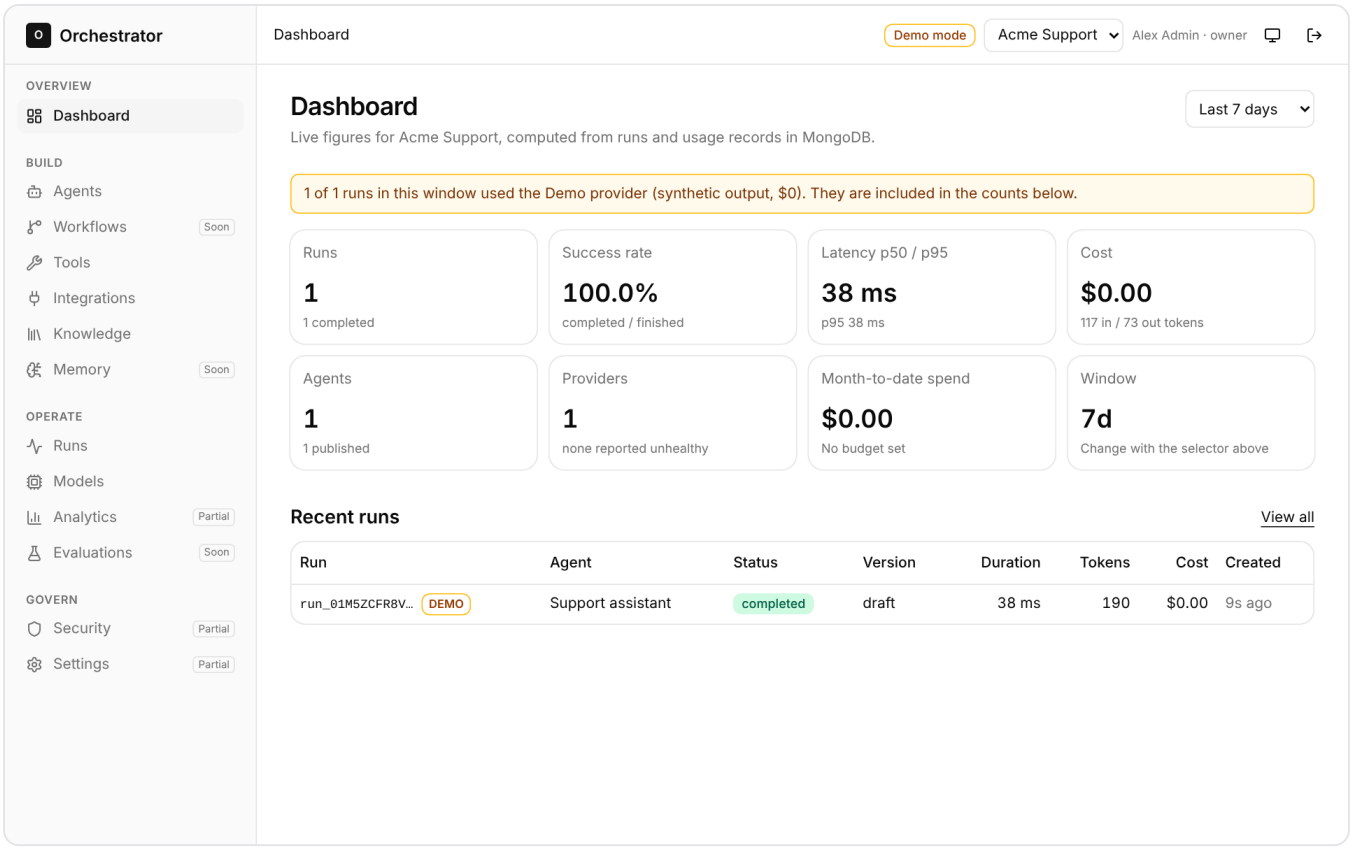
3. Create your account and workspace

Open **Register**, enter your name, email, a password of at least 12 characters, and a name for your first workspace (for example your company or project).



Registration. The first person becomes the owner of the workspace.

After registering you land on the **Dashboard**. It shows runs, success rate, speed and cost for the selected period, and the latest runs.



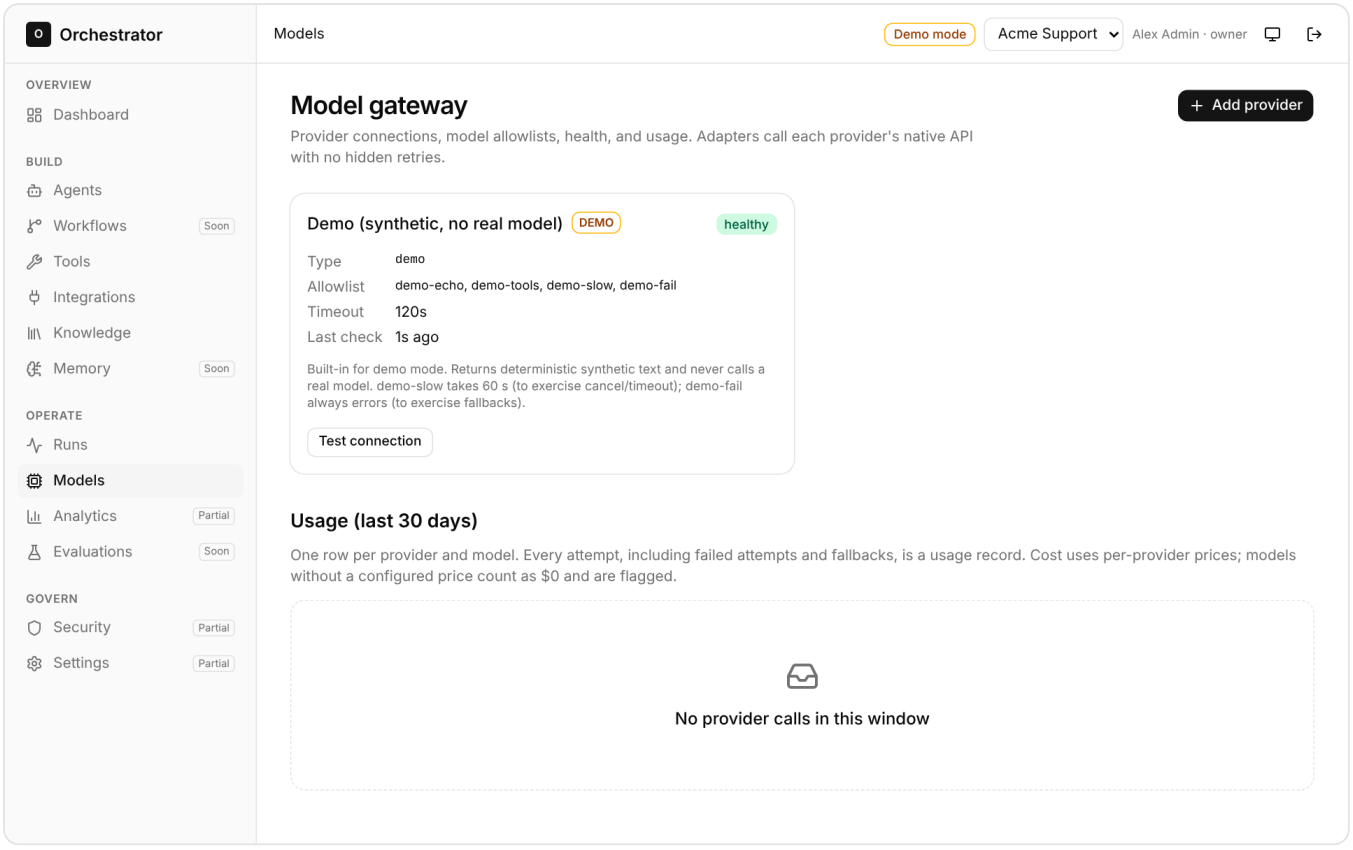
The dashboard. The menu on the left holds every part of the platform. Items marked "Soon" are not built yet.

The box at the top right switches between workspaces. The "Demo mode" badge means the free demo model is available.

4. Connect an AI model

Every workspace starts with a **Demo** provider so you can try everything for free. For real answers, add your own provider:

1. Open **Models** and click **Add provider**.
2. Choose the type: OpenAI, Anthropic, Gemini or OpenAI-compatible. For OpenAI-compatible (Ollama, vLLM, Azure-style gateways, OpenRouter) also enter the base URL.
3. Paste your API key. It is encrypted and never shown again; only the last 4 characters are displayed.
4. Optionally list allowed models and prices per million tokens, so the cost figures are correct.
5. Click **Test connection**. A green "healthy" status means it works.



Models: providers, their health, and usage per model.

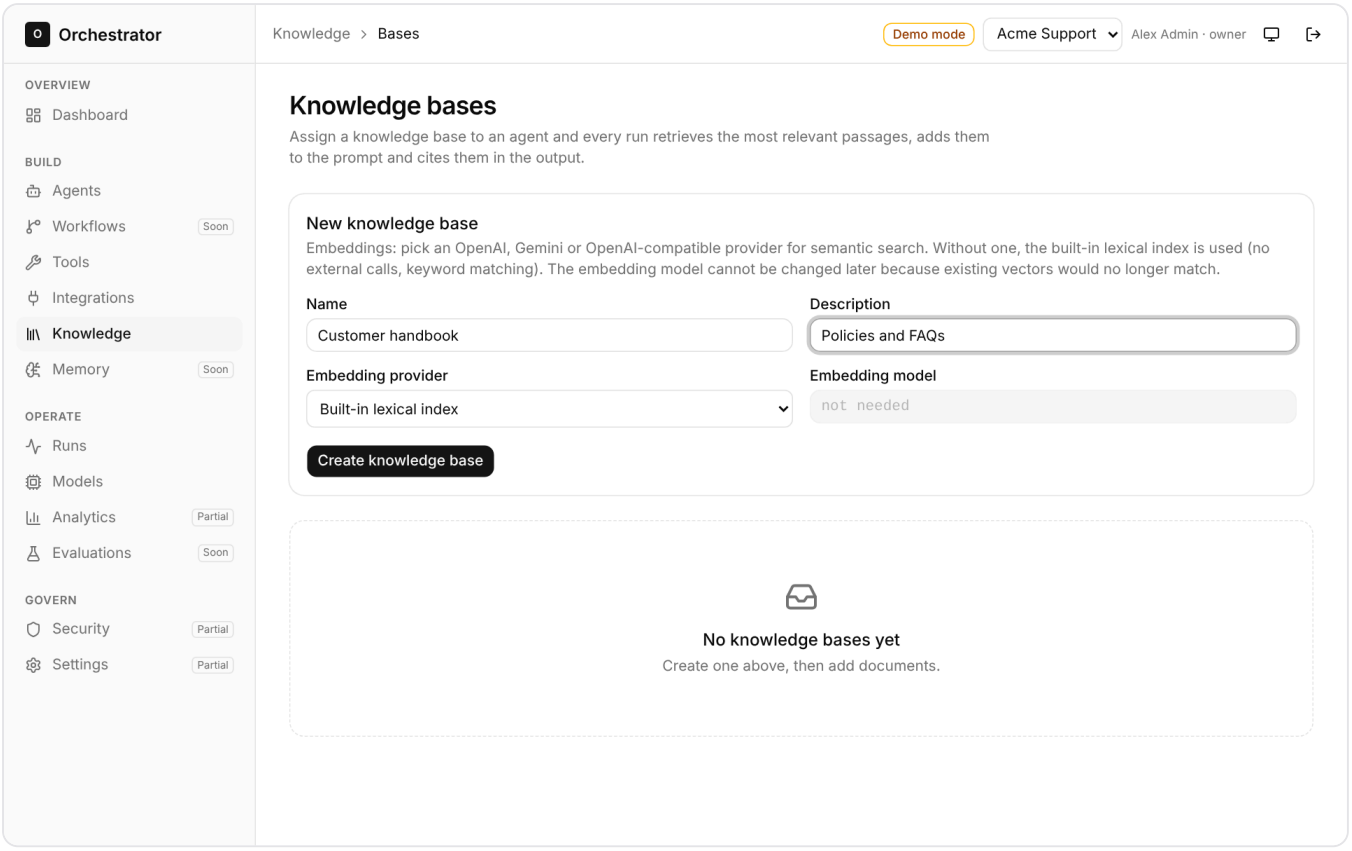
> Only administrators and owners can add providers. Model examples: `gpt-4.1-mini` (OpenAI), `claude-sonnet-4-5` (Anthropic), `gemini-2.5-flash` (Gemini). Use the exact model name from your provider.

5. Add your knowledge (documents)

A knowledge base lets an agent answer from your own content: policies, manuals, FAQs, product sheets.

1. Open **Knowledge → Knowledge bases**. Enter a name.
2. Choose how it searches:
   - **Built-in lexical index** (default): free, offline, matches words. Good for FAQs and policies.
   - **An embedding provider** (OpenAI, Gemini or OpenAI-compatible) for meaning-based search, for example `text-embedding-3-small`. Better when people ask in their own words.

   This choice cannot be changed later, so pick the embedding provider now if you have one.
3. Click **Create knowledge base**.



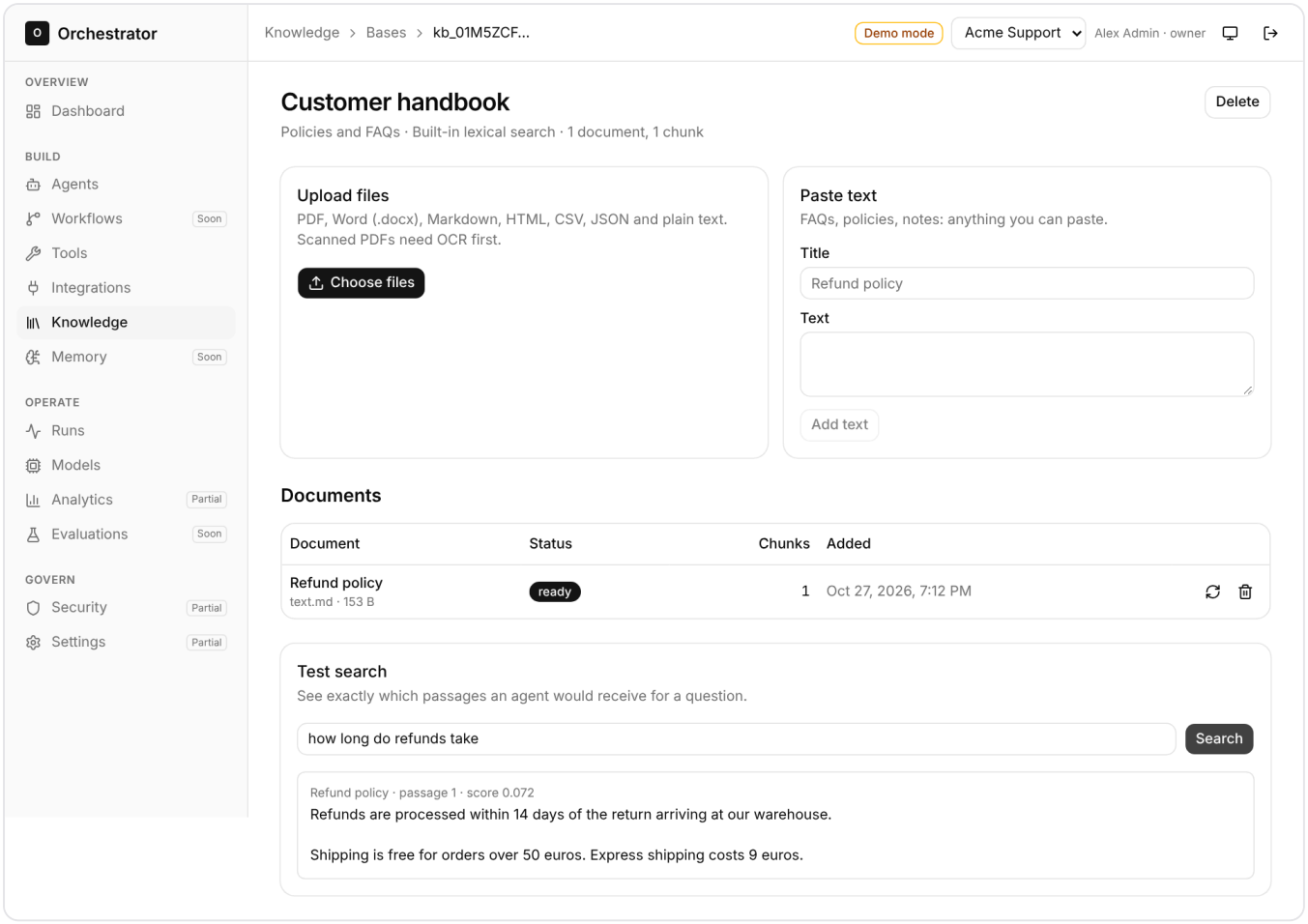
Creating a knowledge base.

Add documents and test the search

- **Upload files:** PDF, Word (.docx), Markdown, HTML, CSV, JSON and plain text, up to 25 MB each. Scanned PDFs (pictures of text) must be OCR-processed first.
- **Paste text:** give it a title and paste the content.

Each document shows a status: `queued` → `processing` → `ready`. The worker reads it, cuts it into passages and indexes them. If it says `failed`, the reason is shown under the title. Use the circular arrow to index it again, or the bin to delete it.

Use **Test search** to type a question and see exactly which passages an agent would receive, with their score.



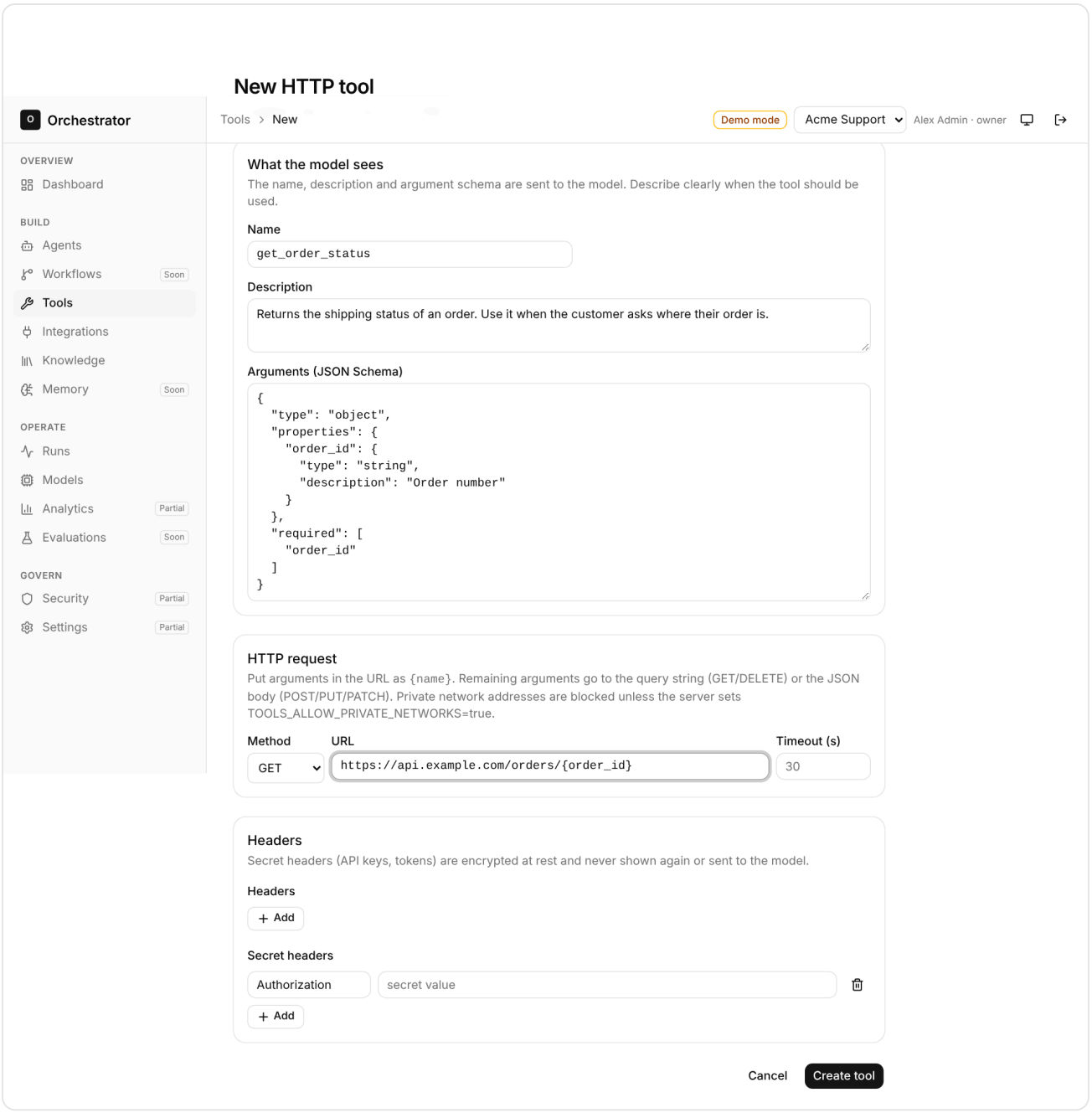
A knowledge base with one ready document and a search test.

6. Add tools (your APIs and MCP servers)

Tools let the agent do things or fetch live data. The model decides when to call a tool, based on its name and description. Every call is checked against the tool's argument rules, run with a time limit, and written to the run's timeline.

An HTTP tool (your own API)

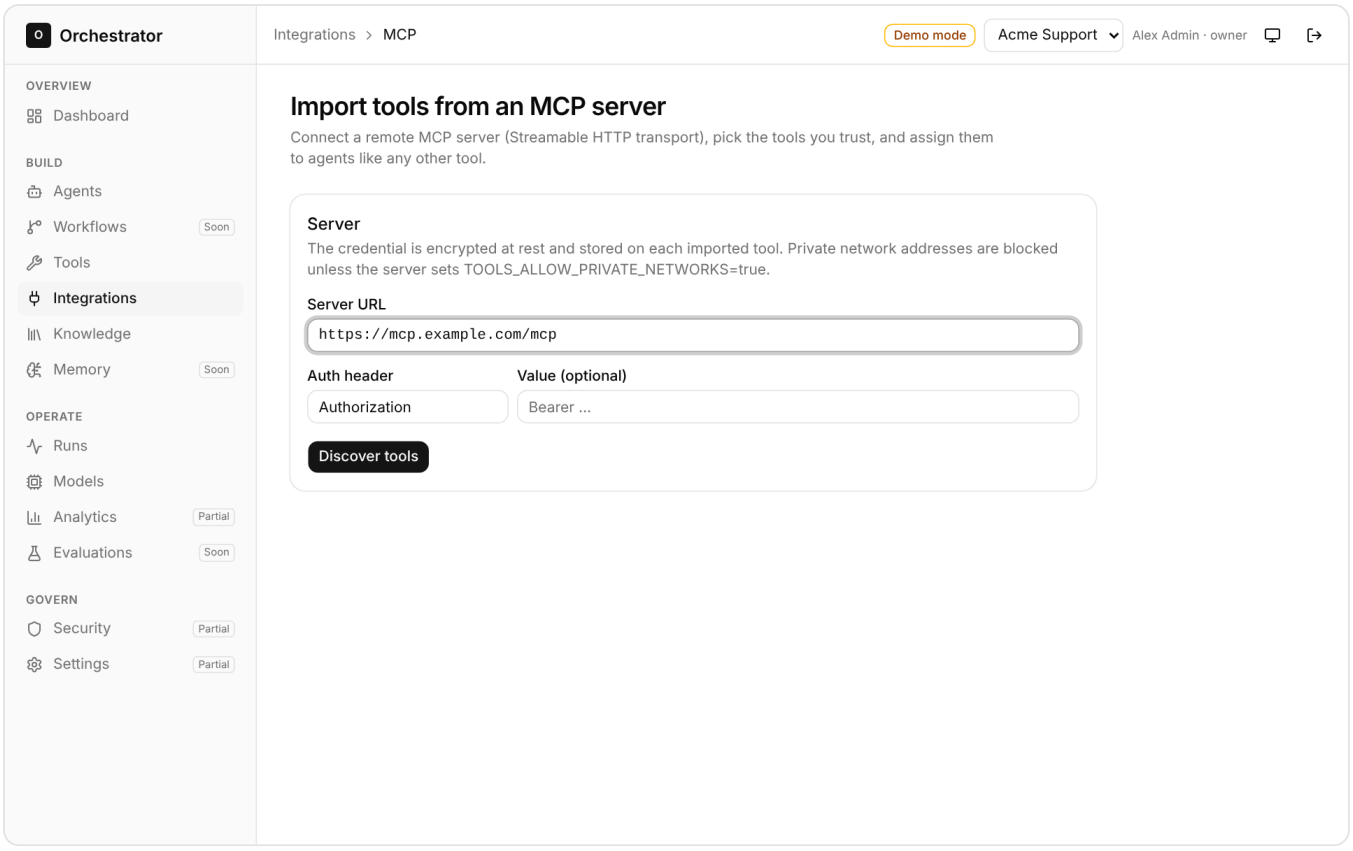
1. Open **Tools → New HTTP tool**.
2. **Name**: short, like `get_order_status`. **Description**: say clearly what it does and when to use it.
3. **Arguments (JSON Schema)**: the inputs. Example: an `order_id` string that is required.
4. **Method and URL**. Put arguments in the URL with braces, like `https://api.example.com/orders/{order_id}`. Other arguments go into the query string (GET, DELETE) or the JSON body (POST, PUT, PATCH).
5. **Secret headers** such as `Authorization: Bearer …` are encrypted and never shown to the model or to users.
6. Save, then use **Test** on the tool page to call it with sample arguments.



Creating an HTTP tool.

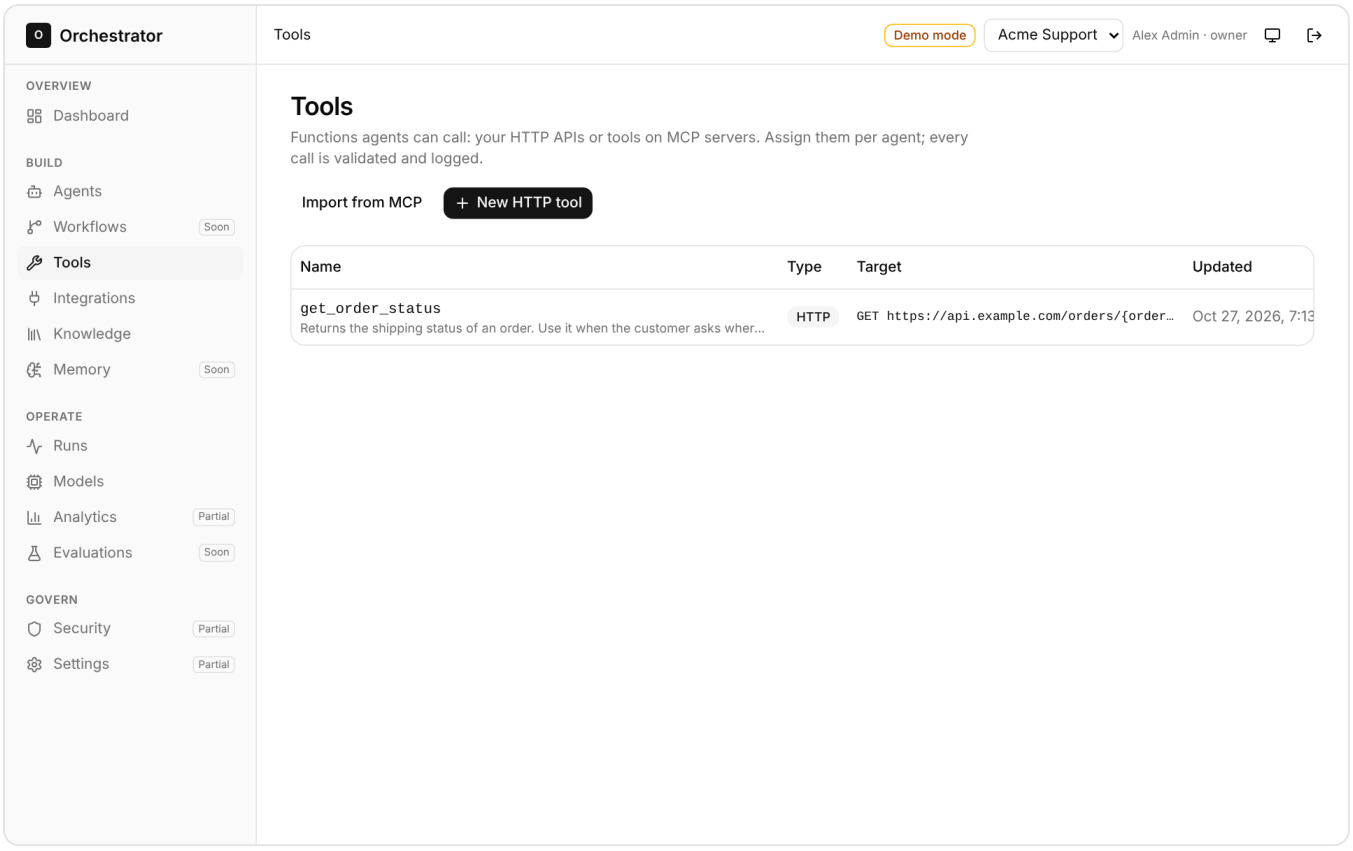
Tools from an MCP server

MCP (Model Context Protocol) servers offer ready-made tools, for example for GitHub, databases or internal systems. Open **Integrations → MCP servers** (or **Tools → Import from MCP**), enter the server URL and, if needed, its token, then click **Discover tools**. Tick the tools you trust, optionally add a name prefix such as `github_`, and click **Import**.



Importing tools from a remote MCP server (Streamable HTTP).

> **Internal addresses are blocked by default.** To protect your network, tools cannot call `localhost`, `10.x`, `192.168.x` and similar addresses. If your API runs inside your own network, set `TOOLS_ALLOW_PRIVATE_NETWORKS=true` in `.env` and restart.



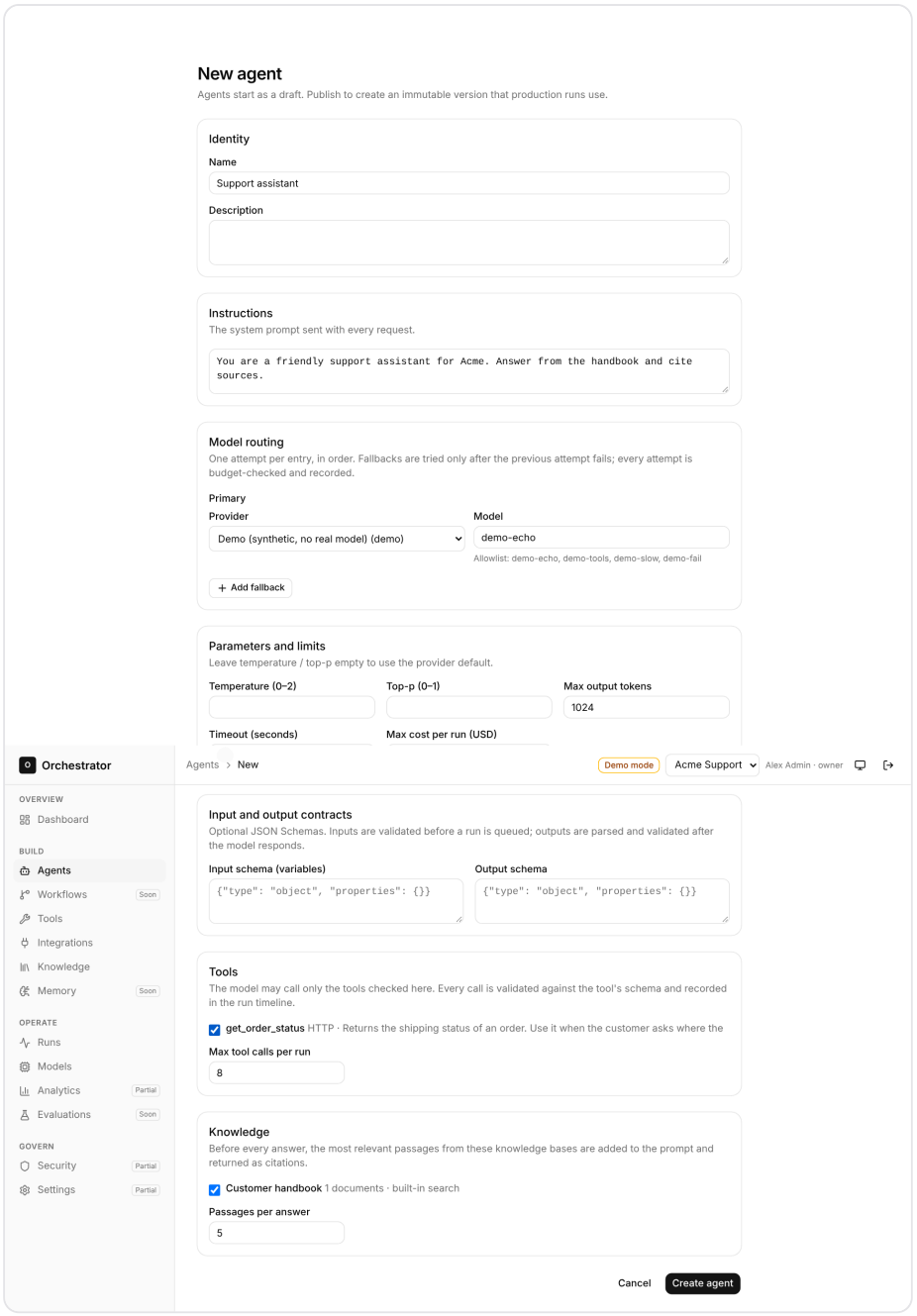
The tools list. Turn a tool off on its page to stop all agents from calling it at once.

7. Build an agent

Open **Agents → New agent** and fill in:

Field	What to put
Name, description	So your team knows what it is for.
System prompt	The instructions: who it is, tone, rules, what to refuse. Example: "You are Acme's support assistant. Answer only from the handbook and cite sources. If you don't know, say so."
Primary model	Provider and model name. For testing, use Demo and `demo-echo`.
Fallbacks	Backup models tried in order if the primary fails (outage, rate limit).
Parameters	Temperature (0 = precise, 1+ = creative), max output tokens.
Tools	Tick the tools this agent may use, and the maximum tool calls per run (default 8).
Knowledge	Tick knowledge bases and how many passages to add to each answer (default 5).
Limits	Timeout, retries, and a cost cap per run.
Output schema	Optional: force answers into a JSON shape, useful when software reads the answer.

Click **Create agent**. It starts as a **draft**.

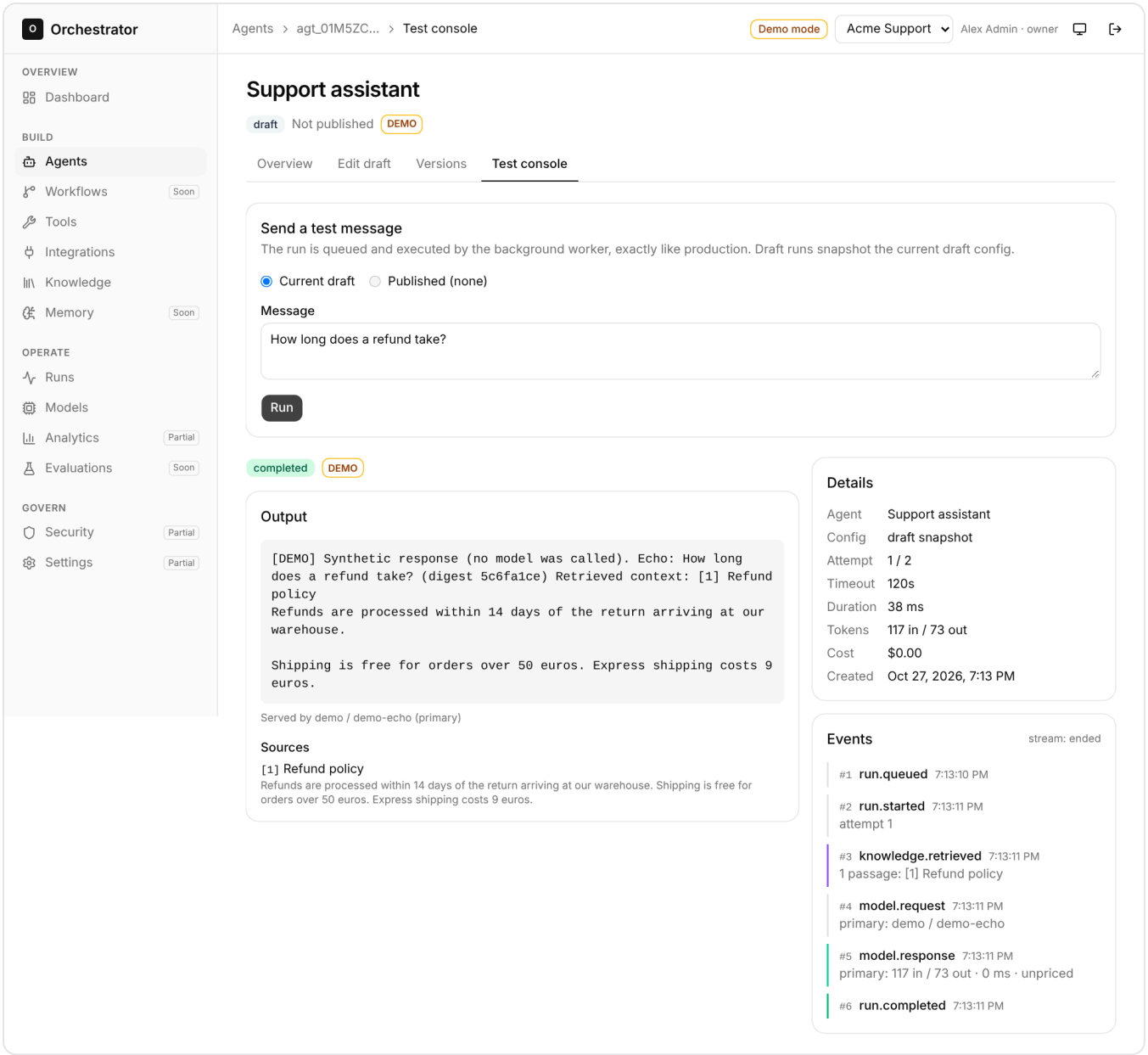


The agent form with a knowledge base and a tool selected.

8. Test, publish and run the agent

Test console

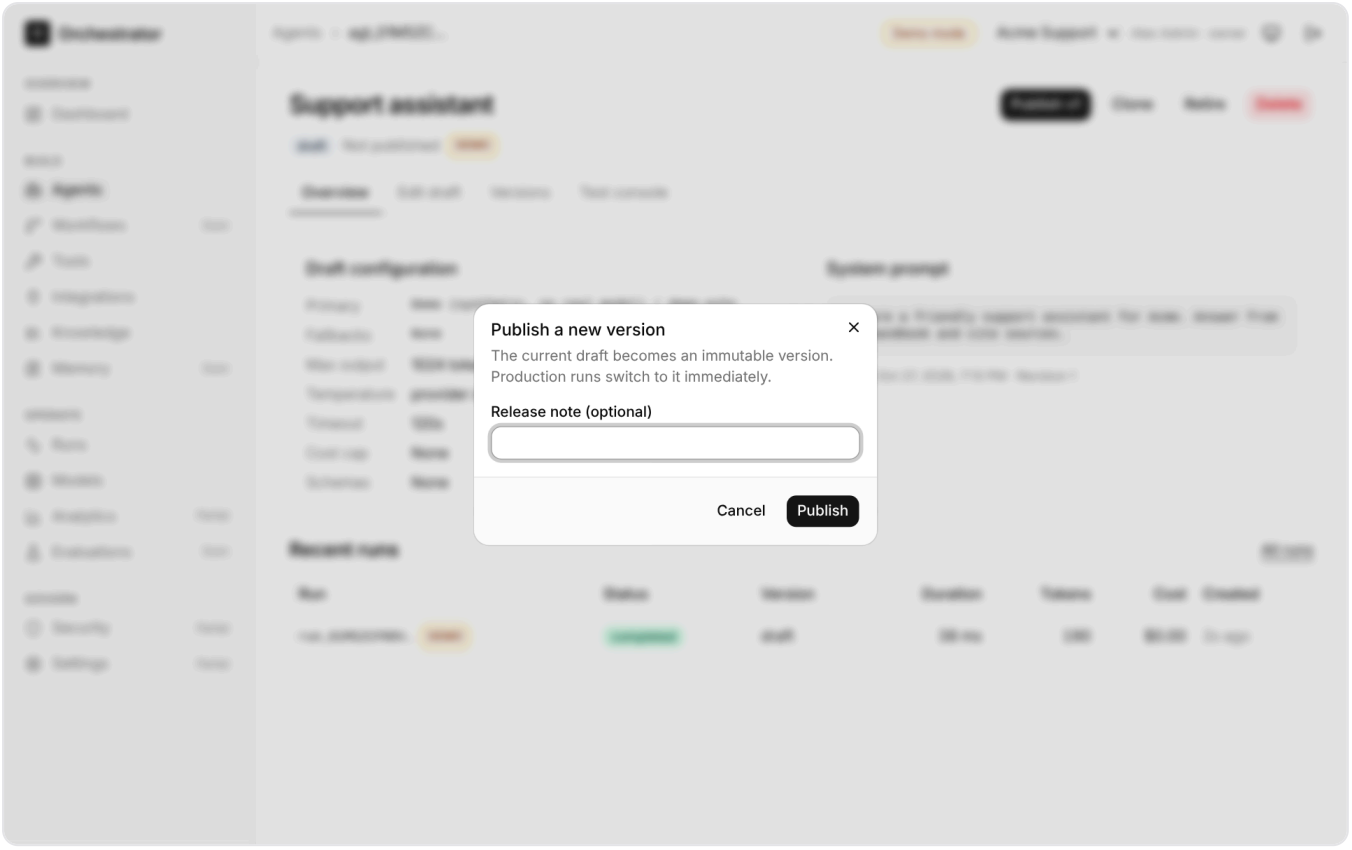
On the agent page open **Test console**, type a message and click **Run**. The run goes through the same queue and worker as production. You see the answer, the **Sources** it used, and a live list of **Events**: knowledge retrieved, model request and response, tool calls and results, completion.



A test run. The demo model echoes the passage it received, which proves retrieval works. A real model writes a proper answer and cites [1].

Publish

When you're happy, click **Publish v1** on the agent's overview and add a note. The published version is frozen. Your apps and operators always use the published version, so you can keep editing the draft safely.

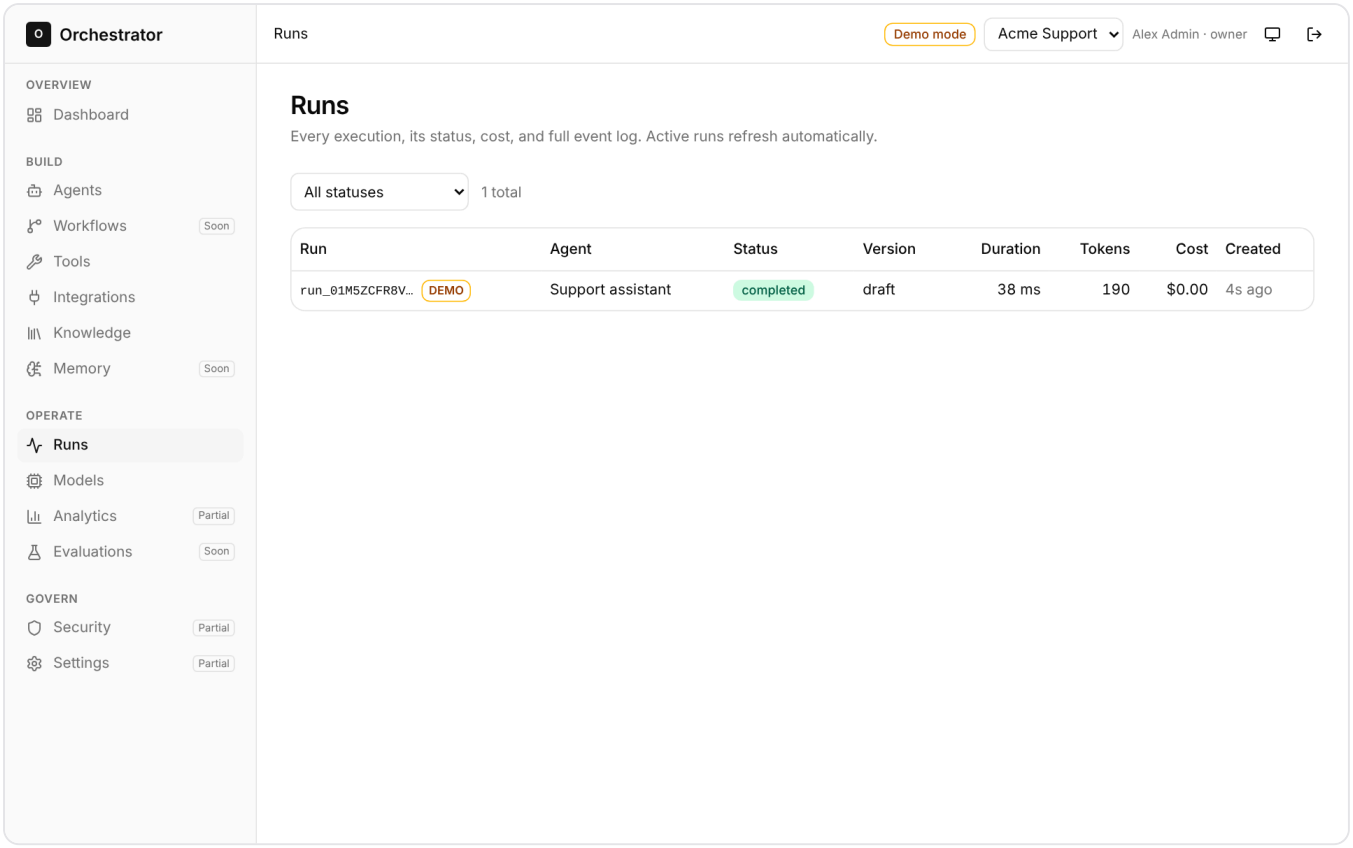


Publishing creates a version that cannot change.

- **Versions** tab: see every version, compare two versions, and **roll back** by publishing an older one again.
- **Pause** stops new runs; **Resume** allows them again; **Retire** ends the agent for good.
- **Clone** copies an agent as a starting point for a new one.

9. Follow runs, costs and the audit log

Runs lists every run with status, agent, duration and cost. Click a run to see its input, output, sources, timeline and the exact configuration it used. Running jobs can be **cancelled**.

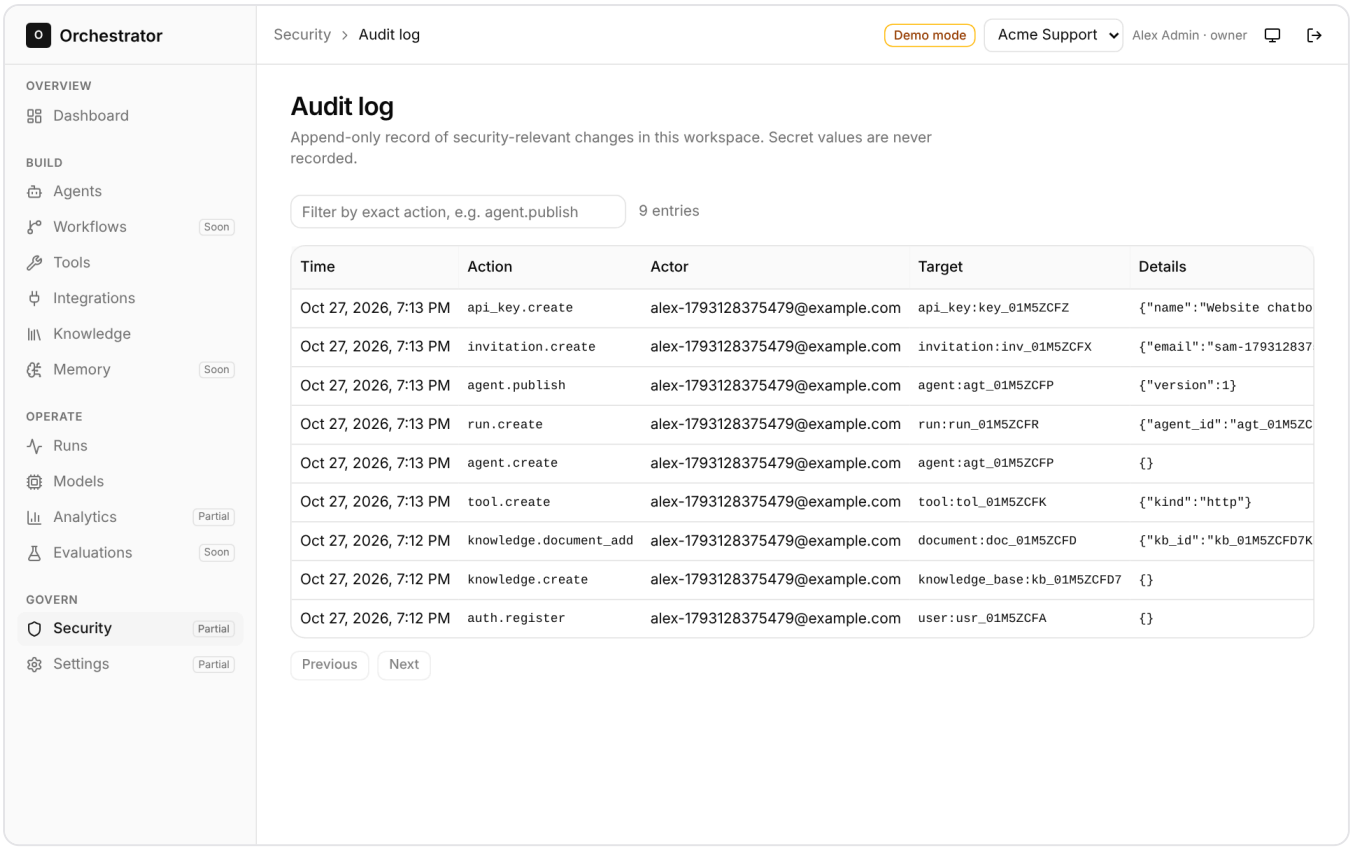


The runs list.

Run statuses: `queued` waiting for a worker · `running` · `completed` · `failed` (the error is shown) · `timed_out` · `cancelled`.

Models shows token usage and cost per provider and model. **Settings → Workspace** lets owners set a monthly budget; runs are refused when it is spent.

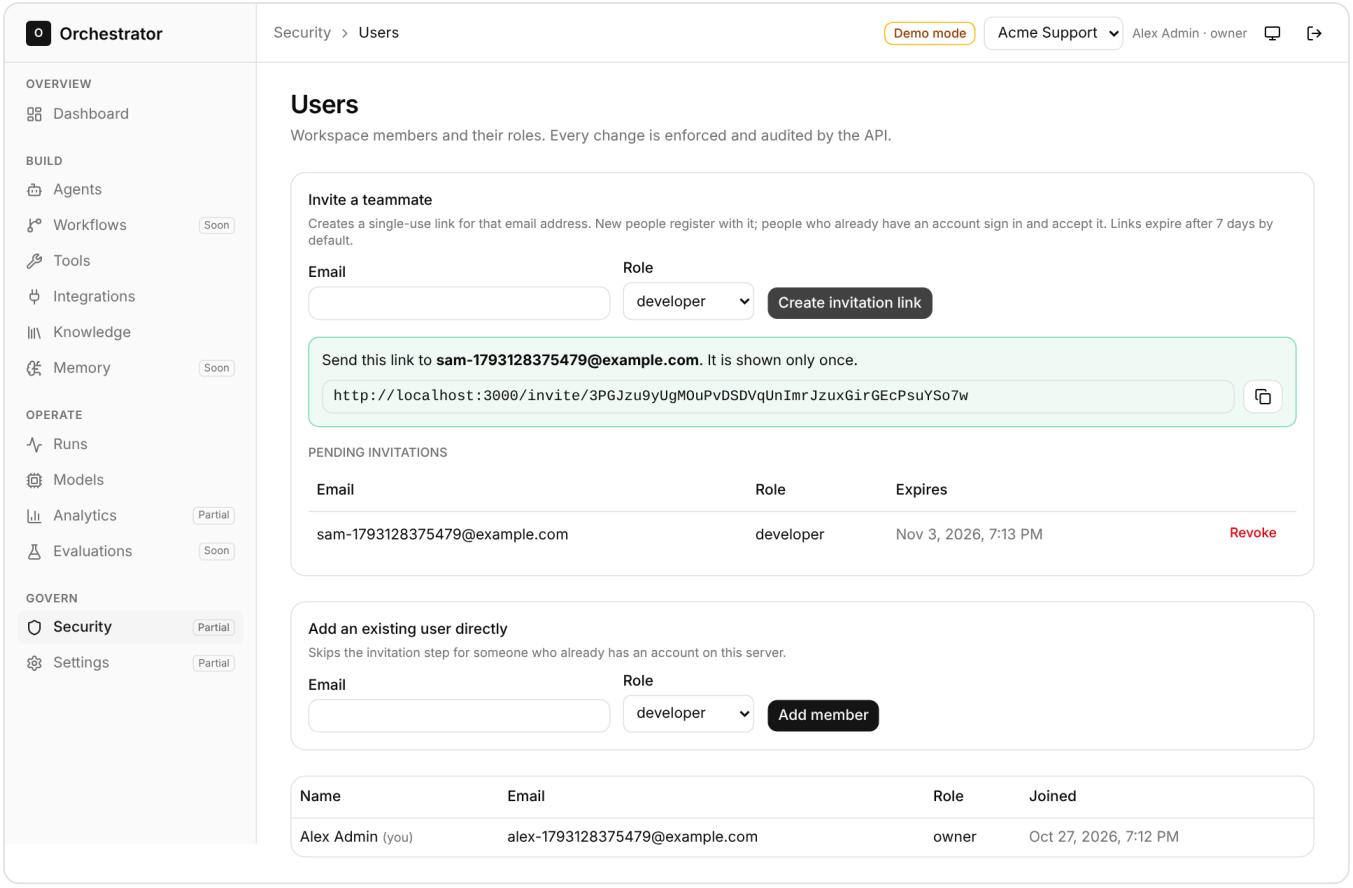
Security → Audit log records who did what and when: publishing, invitations, key changes, tool changes. Secret values are never written to it.



The audit log.

10. Work as a team: invitations and roles

1. Open **Security → Users**.
2. Enter your teammate's email, choose a role, click **Create invitation link**.
3. Copy the link and send it yourself (chat, email). It works once, only for that email, for 7 days.
4. Your teammate opens it, sets a name and password, and lands in your workspace.



Inviting a teammate. Pending invitations can be revoked.

Roles

Role	Can do	Good for
Viewer	See agents, runs, tools, knowledge, usage.	Managers, reviewers
Operator	Viewer + start and cancel runs.	Support staff using agents
Developer	Operator + create, edit and publish agents, tools and knowledge bases.	People who build agents
Administrator	Developer + model providers, members, API keys, workspace settings, audit log.	Team leads, IT
Owner	Everything. Created with the workspace.	You

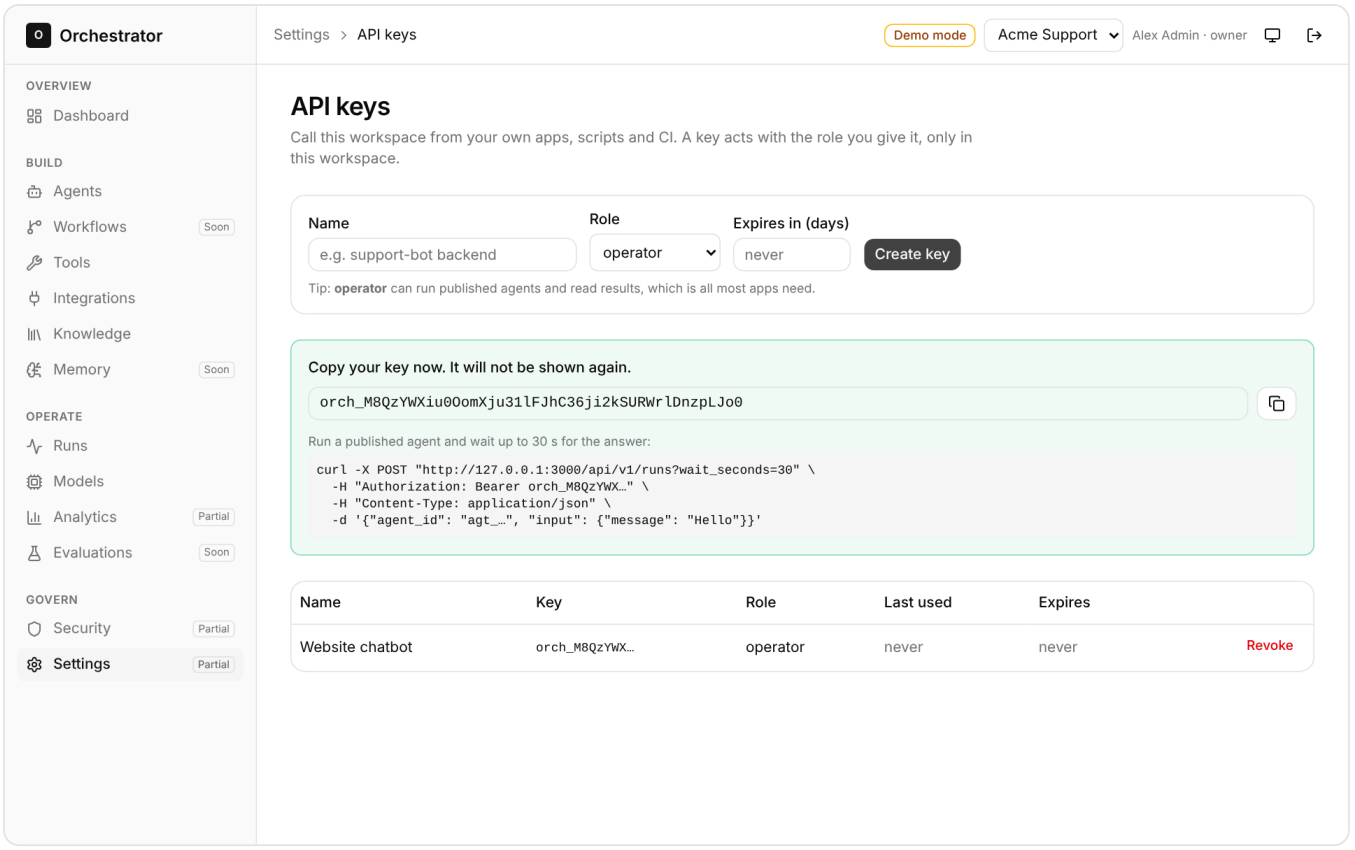
Nobody can give a role higher than their own. Every permission is checked on the server, not only hidden in the screens.

Your own account

- **Settings → Account**: change your password (other sessions are signed out).
- Create another workspace for a separate project from the same page.
- Once your team has joined, turn off open sign-up with `ALLOW_REGISTRATION=false` in `.env`. Invitations still work.

11. Use agents from your own apps (API keys)

1. Open **Settings → API keys**, give the key a name (for example "Website chatbot"), choose a role (operator is enough to run agents) and an optional expiry.
2. Click **Create key** and copy it now. It starts with `orch_` and is shown only once.
3. Copy the agent's ID from its page (it starts with `agt_`). The agent must be published.



A new API key. Revoke it any time; apps using it stop immediately.

Call an agent

Send the message and wait up to 30 seconds for the answer:

```
curl -X POST "http://localhost:3000/api/v1/runs?wait_seconds=30" \
  -H "Authorization: Bearer orch_YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"agent_id": "agt_YOUR_AGENT", "input": {"message": "How long does a refund take?"}}'
```

Python:

```
import httpx

r = httpx.post("https://ai.example.com/api/v1/runs",
               params={"wait_seconds": 30},
               headers={"Authorization": "Bearer orch_YOUR_KEY"},
               json={"agent_id": "agt_YOUR_AGENT", "input": {"message": "Hello"}},
               timeout=40)
run = r.json()
print(run["status"], run["output"]["text"])
for source in run["output"].get("citations", []):
    print(source["n"], source["title"])
```

JavaScript:

```
const r = await fetch("https://ai.example.com/api/v1/runs?wait_seconds=30", {
  method: "POST",
  headers: { Authorization: "Bearer orch_YOUR_KEY", "Content-Type": "application/json" },
  body: JSON.stringify({ agent_id: "agt_YOUR_AGENT", input: { message: "Hello" } }),
})
const run = await r.json()
console.log(run.status, run.output?.text)
```

What comes back

- `status` is `completed` and `output.text` holds the answer. `output.citations` lists sources, `output.tool_calls` lists tool calls, `usage` holds tokens and cost.
- If it is still `queued` or `running` after the wait, ask again with `GET /api/v1/runs/{id}`, or stream live events from `GET /api/v1/runs/{id}/stream`.
- Send an `Idempotency-Key` header with a unique value per request, so a retry never runs the job twice.
- The full API reference is at `/api/docs` on the API (port 8000).

> Keep API keys on your server. Never put them in a web page or mobile app that users can download.

12. Put it on a server for your team

A Linux server with 2 CPUs and 4 GB of memory is enough to start. You need a domain name (for example `ai.example.com`) pointing to the server, with ports 80 and 443 open. The production setup adds Caddy, which gets a free HTTPS certificate automatically.

1. Install Docker on the server and copy the project to `/opt/orchestrator`.
2. Create the settings:

```
cp .env.example .env
./scripts/new-key.sh        # paste the output into ENCRYPTION_KEYS in .env
```

3. In `.env` set `DOMAIN=ai.example.com` and `ACME_EMAIL=you@example.com`. Set `DEMO_MODE=false` if you don't want the demo model.
4. Start:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
docker compose ps           # every service should say "healthy"
```

5. Open `https://ai.example.com`, register the first account, add your model provider, invite your team, then set `ALLOW_REGISTRATION=false` and run the same `up -d` command again.

Important settings in `.env`

Setting	Meaning
`ENCRYPTION_KEYS`	Encrypts stored secrets. Back it up. Required.
`DOMAIN`, `ACME_EMAIL`	Your web address and the email for certificate notices.
`ALLOW_REGISTRATION`	`false` = only invited people can join.
`DEMO_MODE`	Offer the free demo model in workspaces.
`WORKER_REPLICAS`, `WORKER_CONCURRENCY`	How many workers, and runs per worker. Raise for more traffic.
`TOOLS_ALLOW_PRIVATE_NETWORKS`	`true` lets tools call internal addresses.
`TOOL_TIMEOUT_SECONDS`	Time limit for each tool call (default 30).
`MAX_UPLOAD_BYTES`	Largest document upload (default 25 MB).
`INVITATION_TTL_DAYS`	How long invitation links work (default 7).
`COMPOSE_MONGODB_URI`	Use an external MongoDB such as Atlas instead of the built-in one.

Every setting is explained inside `.env.example`. The complete server guide is in `docs/deployment.md`.

13. Everyday operations

Backups

```
./scripts/backup.sh                    # saves backups/<date>.archive.gz, keeps the newest 14
./scripts/restore.sh backups/<file>    # puts a backup back (asks first)
# Daily at 03:00 (crontab -e):
0 3 * * * cd /opt/orchestrator && ./scripts/backup.sh >> backups/backup.log 2>&1
```

On Windows use `./scripts/backup.ps1`. Copy the `backups` folder and `.env` to another place regularly.

Updating to a new version

```
./scripts/backup.sh
# copy the new release over the old folder, keep your .env
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Database changes are applied automatically. Running jobs are finished or handed back safely, so nothing is lost.

Forgotten password (no email server needed)

```
docker compose exec api python -m app.cli reset-password person@example.com
docker compose exec api python -m app.cli list-users
docker compose exec api python -m app.cli create-user EMAIL "Name" --workspace "Team"
```

Logs and health

```
docker compose ps                      # health of every part
docker compose logs -f api worker      # live logs (secrets are hidden)
```

Checking everything works (for developers)

```
./scripts/test.sh           # all automatic tests (Windows: ./scripts/test.ps1)
./scripts/test.sh --e2e     # plus browser tests against http://localhost:3000
```

14. Troubleshooting

Problem	What to do
The page does not open at localhost:3000	Make sure Docker Desktop is running. Run `docker compose ps`; all parts should be "healthy". Look at `docker compose logs api`.
A run stays "queued"	The worker is not running. Check `docker compose ps worker` and `docker compose logs worker`.
Provider test fails	Check the API key and model name. For OpenAI-compatible servers check the base URL (usually ends with `/v1`). Look at the error shown on the provider card.
Run failed with a budget error	The agent's cost cap or the workspace's monthly budget is reached. Raise it in the agent's limits or Settings → Workspace.
A document is "failed"	Read the error under its title. Common causes: scanned PDF without text, unsupported file type, file too large.
The agent does not use my documents	Make sure the knowledge base is ticked on the agent and the documents are "ready". Try the question in Test search. With the lexical index, use words that appear in the documents, or switch to a new knowledge base with an embedding provider.
Tool error "private address"	Your API is on an internal network. Set `TOOLS_ALLOW_PRIVATE_NETWORKS=true` and restart.
"Cannot decrypt" errors after moving servers	The `ENCRYPTION_KEYS` value changed. Restore the original `.env`, or re-enter the provider keys and tool secrets.
Invitation link does not work	It is used, expired, revoked, or opened with a different email. Create a new one.
A teammate cannot see a button	Their role does not allow it. An administrator can change the role in Security → Users.

15. What is ready and what is coming

Area	Status
Accounts, sign-in, password change, workspaces, invitations, five roles	Ready
Model providers: OpenAI, Anthropic, Gemini, OpenAI-compatible, fallbacks, cost tracking, budgets	Ready
Agents: drafts, versions, publish, compare, rollback, pause, retire, clone, test console	Ready
Runs: queue, worker, live events, cancel, timeouts, retries, crash recovery	Ready
Tools: HTTP APIs and MCP servers	Ready
Knowledge: documents, search, sources in answers	Ready
API keys and calling agents from your own code	Ready
Production: HTTPS, health checks, backups, automatic tests	Ready
Dashboard, analytics, audit log	Ready (analytics is basic)
Workflows (chaining several agents)	Coming
Memory (agents remembering past conversations)	Coming
Evaluations (testing answer quality on a set of questions)	Coming
Billing, single sign-on (SSO), two-factor login, email sending	Coming

Screens for parts that are coming are marked "Soon" in the menu and contain no buttons that pretend to work.